User Manager: Next.js + Express + Prisma (SQLite and PostgreSQL)

A full-stack learning project: a single-page Next.js app that lists, adds, edits, and deletes users by calling an Express REST API. The backend is built twice, once with SQLite and once with PostgreSQL in Docker, so you can compare both. The frontend is identical for both backends.

Repository Structure

```
user-manager/
  frontend/              Next.js app (port 3000)
  backend/               Express + Prisma + SQLite (port 3001)
  backend_postgresql/    Express + Prisma + PostgreSQL in Docker (port 3001)
  README.md
```

Run only one backend at a time, because both use port 3001.

Tech Stack

Part	Technology
Frontend	Next.js 16 (App Router), TypeScript, plain CSS, fetch
Backend	Node.js, Express, TypeScript run by tsx, cors, dotenv
ORM	Prisma 7 with a driver adapter
Database A	SQLite (a single file, dev.db) using @prisma/adapter-better-sqlite3
Database B	PostgreSQL 16 using @prisma/adapter-pg and pg
Containers	Docker, Dockerfile, and Docker Compose (PostgreSQL backend)

How It Fits Together

- **SQLite flow**: Node.js -> Prisma -> adapter -> SQLite file
- **PostgreSQL flow**: Node.js -> Prisma -> adapter -> pg -> PostgreSQL database
- **Frontend**: browser -> Next.js page -> fetch -> Express API -> database

The frontend only talks to the Express API, so it needs no changes when the database changes.

Features

- List all users in a table
- Add a user (name and email)
- Edit a user
- Delete a user
- Duplicate email shows a red error message (HTTP 409)

Prerequisites

- Node.js 20 or newer and npm
- Git
- Docker Desktop (only for the PostgreSQL backend)
- Postman (optional, for testing the API)

Quick Start A: SQLite Backend

```
cd backend
npm install
npx prisma migrate dev --name init
npx prisma generate
npm run dev
```

Create `backend/.env` first:

```
DATABASE_URL="file:./dev.db"
PORT=3001
```

The server starts on http://localhost:3001. Open http://localhost:3001/users to check it.

Quick Start B: PostgreSQL Backend With Docker Compose

This is the easiest way. One command starts PostgreSQL and the backend together.

```
cd backend_postgresql
docker compose up --build
```

The Dockerfile applies the saved migrations at startup, so the User table is created automatically. When you see "Server running on http://localhost:3001", open http://localhost:3001/users. An empty array means it works.

To stop everything:

```
docker compose down
```

The data stays in the pgdata volume. `docker compose down -v` also deletes the volume and wipes the database.

Quick Start C: PostgreSQL Backend Without Compose (local development)

Use this when you want npm run dev with auto-restart while coding.

1. Start only the database:

```
docker run --name usersdb -e POSTGRES_USER=postgres -e
POSTGRES_PASSWORD=postgres -e POSTGRES_DB=usersdb -p 5432:5432 -d postgres:16
```

2. Create `backend_postgresql/.env`:

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/usersdb"
PORT=3001
```

3. Install, migrate, and run:

```
cd backend_postgresql
npm install
npx prisma migrate dev --name init
npx prisma generate
npm run dev
```

If port 5432 is already used, change only the left number, for example `-p 5433:5432`, and use 5433 in DATABASE_URL.

Run the Frontend

1. Start one backend first (port 3001).
2. Create `frontend/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:3001
```

3. Install and run:

```
cd frontend
npm install
npm run dev
```

4. Open http://localhost:3000.

Next.js reads .env.local only at startup, so restart npm run dev after changing it.

API Endpoints

Method	Endpoint	Description	Success status
GET	/users	List all users	200
GET	/users/:id	Get one user	200 (404 if missing)
POST	/users	Create a user (body: name, email)	201 (409 if email exists)
PUT	/users/:id	Update a user (body: name, email)	200
DELETE	/users/:id	Delete a user	204

Example body for POST and PUT:

```
{
  "name": "King",
  "email": "king@example.com"
}
```

Testing

Postman: GET /users returns an empty array. POST a user with raw JSON, GET it back, PUT a changed name, DELETE it (204), and POST the same email twice to see 409.

Frontend flow: add a user and see the row appear, click Edit and Update, click Delete, and add a duplicate email to see the red error.

SQLite vs PostgreSQL

	SQLite backend	PostgreSQL backend
Where data lives	One file (dev.db)	A database server in Docker
Adapter packages	@prisma/adapter-better-sqlite3	@prisma/adapter-pg and pg
Extra software	None	Docker Desktop
DATABASE_URL	file:./dev.db	postgresql://user:password@host:5432/dbname
Prisma provider	sqlite	postgresql
Best for	Quick learning, no setup	Closer to production

The User model, the routes, and the frontend are the same in both.

Troubleshooting

- **Failed to fetch in the frontend**: the backend is not running, or .env.local has the wrong address. Restart the frontend after editing .env.local.
- **Port 3001 already in use**: stop the other backend. Only one can run at a time.
- **Port 5432 already in use**: change the left number of the port mapping.
- **Cannot connect to database**: check that Docker Desktop is running and that DATABASE_URL matches your Docker values.
- **Cannot GET /**: expected. Open /users instead.
- **Prisma update available message**: harmless.

- **Docker install error about Virtual Machine Platform (Windows)**: turn on Virtual Machine Platform and Windows Subsystem for Linux in Windows features, run `wsl --install` in an administrator terminal, and restart.

Important Notes for GitHub

- Never commit `.env` files, `node_modules`, the generated Prisma client, or `dev.db`. The repository's .gitignore excludes them.
- Anyone who clones the project creates their own .env files, using the examples above.
- The passwords in this README are for local learning only. Use strong random passwords for any real server.

Deployment

The PostgreSQL backend is ready to deploy with Docker Compose on a VPS, or to use a managed database such as Neon, Supabase, or AWS RDS. See the separate deployment guide for the steps.

What You Learn From This Project

- How a frontend, an API, and a database fit together
- What an ORM, a migration, and a driver adapter are
- How the same code works with SQLite and PostgreSQL
- What a Dockerfile, Docker Compose, a port mapping, and a volume do
- How to test an API in Postman

License

For learning. Use and modify freely.